Klotski v2: Improved DNN Model Orchestration Framework for Dataflow Architecture Accelerators

Chen Bai, *Member, IEEE*, Xuechao Wei, Youwei Zhuo, *Member, IEEE*, Yi Cai, Hongzhong Zheng, *Member, IEEE*, Bei Yu, *Senior Member, IEEE*, and Yuan Xie

***Abstract*—Dataflow architecture accelerators are a new kind of scalable DNN accelerators. For an instruction, the availability of input operands solely determines the beginning of executions. DNN model orchestration determines how to partition, schedule, and map the computation to the underlying hardware. In this article, we propose the Klotski v2 framework to solve DNN model orchestration for dataflow architecture accelerators. First, a Bayesian optimization-based entropy-directed partition algorithm is proposed to transform a DNN model into μops. Second, a unified formal formulation for μops scheduling and mapping is presented. Third, a two-stage methodology is proposed to decouple the scheduling and mapping. Fourth, a Hilbert curve-based mapping heuristic is proposed to enhance problem-solving efficiency, improving the tradeoff between solution quality and algorithm runtime. Extensive results show that Klotski v2 can achieve an average of 21.57% higher execution performance improvement than the previous methodologies. With the Hilbert curve-based mapping heuristic, we improve the algorithm efficiency by an average of 63.50% across different DNN workloads.**

***Index Terms*—AI accelerators, partitioning algorithms, scheduling algorithms.**

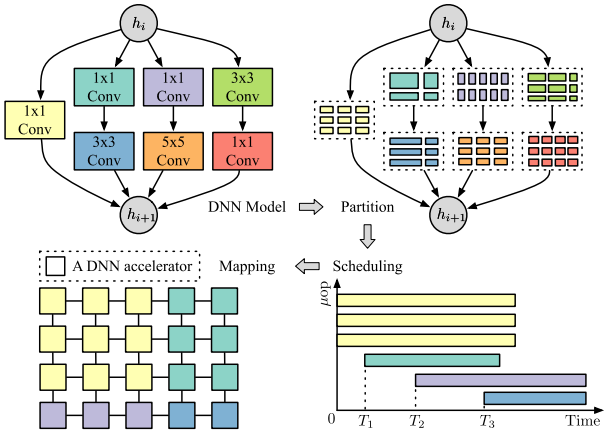


Fig. 1. Pipeline overview of DNN model orchestration for scalable DNN accelerators connected using the mesh topology. The example uses the GoogLeNet inception module. h_i and h_{i+1} are for hidden inputs and outputs. Partition generates different shapes of μ*tensors* for μ*ops*. Scheduling allocates μops' *time slots*. Mapping assigns accelerators for μops. Multiple μops can be executed simultaneously.

I. Introduction

THE DNN models continue to evolve, hit breakthroughs, and gain astonishing outcomes with emergent abilities [1]. The success is achieved via computational scaling and algorithm innovation, which incurs deeply stacked layers and complicated model structures and connections [2], [3], [4].

The ultimate pursuit of extremely efficient DNN model inference propelled the appearance of various DNN accelerators [5], [6], [7], [8]. And the DNN accelerators are also scaled to keep pace with the increasing DNN models' size and structural complexity. The accelerator scales following two aspects. First, a DNN accelerator becomes more "brawny" [5], [8]. The hardware resources, like on-chip memory, functional units, etc., are assigned more. Second, when brawny scaling cannot offer more profits from performance improvements due to the power wall or memory wall, independent DNN accelerators are connected via a dedicatedly designed network-on-chip (NoC) [9], [10], [11], [12], [13]. As a result, they formulate scalable DNN accelerators that we term scalable scaling. The scalable scaling is effective. The philosophy behind this is that collaborative multiprocessing opens opportunities for exploiting higher execution parallelism.

Dataflow architecture accelerators are a new kind of scalable DNN accelerators [14], [15], [16], [17]. A fundamental distinction from the previous scalable DNN accelerators is the execution model [18], [19]. Namely, in dataflow architecture, the executability and execution of instructions is solely determined based on the availability of input operands to the instructions [19], [20]. In other words, dataflow architecture facilitates the asynchronous mechanism where multiple instructions operate on multiple data streams simultaneously (MIMD). Fine-grained parallelism is allowed, and most synchronization steps are removed compared to traditional scalable DNN accelerators [11].

The orchestration of DNN models determine how to partition, schedule, and map the model to scalable DNN accelerators, as shown in Fig. 1. Previous works propose corresponding solutions [11], [21], [22], [23], [24]. Convolutional

Manuscript received 20 April 2024; revised 4 July 2024; accepted 12 August 2024. Date of publication 21 August 2024; date of current version 21 February 2025. The work of Chen Bai was supported in part by the Internship at Alibaba DAMO Academy, and in part by the AI Chip Center for Emerging Smart Systems (ACCESS) and Research Grants Council of Hong Kong SAR under Grant CUHK14210723 and Grant CUHK14211824. This article was recommended by Associate Editor N. K. Jha. *(Corresponding authors: Xuechao Wei; Bei Yu.)*

Chen Bai and Bei Yu are with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong, SAR, China (e-mail: byu@cse.cuhk.edu.hk).

Xuechao Wei, Yi Cai, and Hongzhong Zheng are with the Computing Technology Laboratory, Alibaba DAMO Academy, Hangzhou 311121, China (e-mail: xuechao.wxc@alibaba-inc.com).

Youwei Zhuo is with the School of Integrated Circuits, Peking University, Beijing 100871, China.

Yuan Xie is with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, SAR, China.

Digital Object Identifier 10.1109/TCAD.2024.3446858

1937-4151 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

neural network (CNN)-Partition proposed a hardware resource partition method to maximize the efficiency of accelerators [21]. Tangram applied alternate layer loop ordering (ALLO) dataflow to improve the interlayer parallelism and leveraged the zig-zag mapping strategy to the underlying accelerators [11]. Atomic dataflow partitioned DNN models with simulated annealing and scheduled the DNN computation graph in finer granularity with dynamic programming and heuristics [22]. SET adopted simulated annealing to explore the interlayer scheduling space [23].

Nevertheless, these methods are proposed for traditional scalable DNN accelerators instead of dataflow architecture accelerators. In this article, we propose Klotski v2, a DNN model orchestration framework for dataflow architecture accelerators. The highlights of Klotski v2 are as follows. First, we propose a Bayesian optimization-based (BO) entropy-directed DNN partition algorithm. We flatten a DNN model to many μops (shortened from "micro-operations") and turn layer connections into small computing patterns. Thus, the concept of interlayer and intralayer (or interoperator and intraoperator in other literature) parallelism is giving way to the inter-μops and intra-μops parallelism. Second, we provide a unified formal integer programming formulation for scheduling and mapping. Third, we propose a two-stage methodology to decouple the scheduling and mapping, making the solution feasible. Specifically, the scheduling allocates time slots for each μop to attain the promising *makespan*. And the mapping decides the accelerator allocation for a μop. Our mapping algorithm can minimize the NoC transfer overheads. Fourth, we propose a Hilbert curve-based mapping heuristic to speed up the solution with the two-stage methodology. The heuristic is proposed based on the interpretation of the mapping formulation. And it enhances Klotski v2 and improves the tradeoff between the solution quality and algorithm efficiency.

Our contributions are summarized as follows.

1) A BO-based entropy-directed partition algorithm is proposed for μops generation.
2) A unified formal formulation for the scheduling and mapping is proposed for the newly emerged dataflow architecture accelerators.
3) A two-stage methodology decoupling the unified formulation is proposed to make the solution feasible. With the methodology, we can minimize the makespan and NoC communication overheads.
4) A Hilbert curve-based mapping heuristic is proposed to speed up the two-stage methodology solution.
5) Extensive results with the in-house simulator show that Klotski can achieve an average of 21.57% higher execution performance improvement than two baselines. With the Hilbert curve-based mapping heuristic, we improve the efficiency of Klotski by an average of 63.50% across different DNN workloads.

The remainder of this article is organized as follows. Section II introduces the preliminaries and the problem formulation. Section III details the Klotski framework. Section V is for experiments. Section IV presents the mapping heuristic. Finally, Section VI concludes this article.

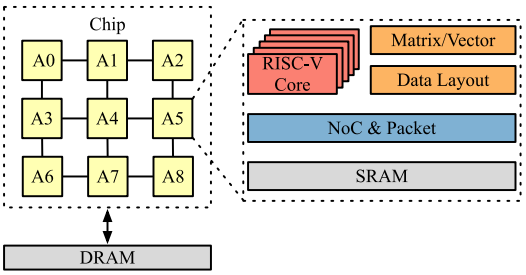


Fig. 2. Overview of dataflow architecture accelerators.

II. Preliminaries

This section provides a background of dataflow architecture accelerators (Section II-A) and two basic scheduling techniques that Klotski v2 bases on (Section II-B). We also present the problem formulation (Section II-C).

A. Dataflow Architecture Accelerators

The traditional scalable DNN accelerators tile individual accelerators in a 2-D manner, and main memories surround accelerators [6], [8], [11], [25]. A microprocessor is leveraged to trigger the execution of scalable DNN accelerators. It sends instructions, controls data movement, and synchronizes all DNN accelerators.

Similarly, an alternative implementation of dataflow architecture accelerators uses an NoC to connect independent accelerators, but with some architecture revisions [16], as shown in Fig. 2. Each accelerator numbered from "A0" to "A8" works asynchronously, and no central governor is incorporated. Multiple microprocessors, e.g., RISC-V cores, are integrated into each accelerator to facilitate instruction coordination. "NoC and packet" shown in Fig. 2 is responsible for flow control, backpressure, and enabling dataflow execution. The computation is purely triggered by producer-consumer relations. The instruction set architecture (ISA) specifies the target locations to store results while neglecting locations to read source operands [19], [20]. To some extent, dataflow execution can be viewed as an out-of-order microexecution with an unlimited instruction issue window size. Such a mechanism relies on communication via a proprietary NoC [16], [17], [26]. Therefore, the primal difference between the dataflow architecture accelerators and the traditional scalable DNN accelerators leads to the existence of accelerator synchronization. Tensor computation is scheduled per *round* in the traditional scalable DNN accelerators. Instead, the dataflow architecture accelerators schedule a tensor computation at any time slot without synchronization.

Fig. 3 details the difference. The computation of convolution layers is from Fig. 1 with the same color legend. All layers are partitioned into μops, and arrows specify the dependencies. For instance, μop 4-1 relies on the results of μop 1-1, 2-1, 3-1, and 3-2. We compare the computation of these μops with traditional scalable DNN accelerators and dataflow architecture accelerators, each involving four individual accelerators. In traditional scalable DNN accelerators, μops are scheduled per round. Synchronization latencies are produced between

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

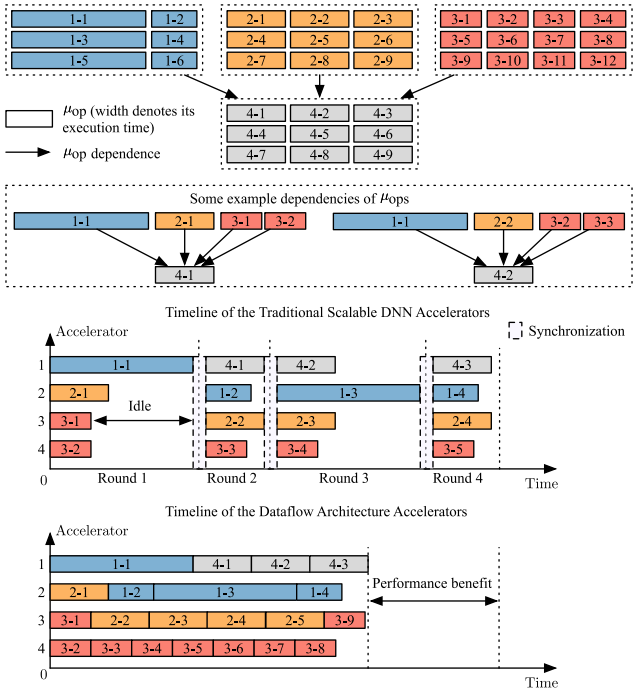


Fig. 3. Comparison between traditional scalable DNN accelerators and dataflow architecture accelerators.

adjacent rounds. The time interval of a round depends on μop with the highest execution latency in that round. Oppositely, dataflow architecture accelerators permit exceptionally higher performance efficiency. The performance benefits come from two aspects. First, the synchronization overheads are eliminated, and a μop can be scheduled for any time slot, such as when two designs accomplish the computation of μop 4-3. Second, the μop processing throughputs are highly increased. Such as the accelerator 4 in dataflow architecture can aggressively be filled with more μops from the red convolution layer.

B. ASAP and ALAP Scheduling

As soon as possible (ASAP) and as late as possible (ALAP) algorithms are two basic scheduling techniques. By combining two algorithms, the scheduling flexibility of each μop can be attained, i.e., the scheduling flexibility hints at the earliest and the latest scheduled time slot for μops.

Fig. 4 gives the overview of ASAP and ALAP scheduling results with 12 μops (suppose the computation latency of each μop equals one). μOp 3 has scheduling flexibility between the time slots 1 and 2, and the μop 8 is between time slots 2 to 4.

Formally, the scheduling flexibility of a μop i given by ASAP and ALAP is formulated as follows:

$$\mathrm{ASAP}(i) = \begin{cases} \max \mathrm{ASAP}(j) + l(j), & \exists\, j \prec i \quad \forall j \in V \\ 0, & \nexists\, j \prec i \quad \forall j \in V \end{cases} \tag{1}$$

$$\mathrm{ALAP}(i) = \begin{cases} \min \mathrm{ALAP}(j) - l(j), & \exists\, i \prec j \quad \forall j \in V \\ \max_{k \in V} \mathrm{ALAP}(k), & \nexists\, i \prec j \quad \forall j \in V. \end{cases} \tag{2}$$

In (1) and (2), V is the set of μops, and $l(j)$ is the execution latency of μop j. $j \prec i$ denotes producer-consumer relations. j is the producer and i is an immediate consumer.

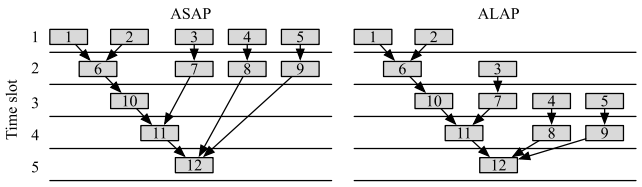


Fig. 4. Overview of ASAP and ALAP scheduling.

C. Problem Formulation

We flatten layers of a DNN model into μops. We use a directed acyclic graph (DAG) $G(V, E)$ to represent the DNN computation for dataflow architecture accelerators. V denotes the set of all μops (as in Section II-B), and E represents the producer and consumer relations between these μops. We first introduce definitions. Then, we formally give three problem formulations for DNN model orchestration.

Definition 1 (μOp): μOp is defined as the execution granularity of an individual accelerator in dataflow architecture accelerators.

A μop's operands are termed μtensors. The execution granularity is the minimal scheduling unit. For example, the execution granularity can be per layer, per-atomic dataflow [22], etc., for a DNN model.

Property 1 (μOp Precedence Constraints): The consumer μop should not begin to execute before the producer μops are completed.

Following Section II-B, suppose i and j are two different μops, and $i \prec j$. So, the earliest possible time slot for j to start execution is when all its dependent μops are finished execution according to Property 1.

Definition 2 (Accelerator): An accelerator executes one μop at a time until completion. Other μops cannot preempt the execution.

We denote the dataflow architecture accelerators as $\boldsymbol{a}$, where $\boldsymbol{a} = \{a_1, a_2, \ldots, a_m\}$, each element of which refers to an accelerator defined in Definition 2.

The orchestration of a DNN model for dataflow architecture accelerators is formulated into three problems.

Problem 1 (Partition): The partition problem is to partition the computation of a DNN model into μops, aiming to maximize utilization of each accelerator, and achieve load balancing, given a set of constraints.

Partitioning the computation of the DNN model into μops can introduce different parallelism granularity, as indicated by Definition 1. Finer-grained parallelism is achieved by partitioning the computation with smaller μops.

Problem 2 (Scheduling): The scheduling problem is to solve the allocation of time slots for μops (the solution from Property 1), aiming to minimize the makespan.

The scheduling problem decides the time slot allocations for each μop Property 1. The mapping decides which individual accelerator is assigned to a given μop.

Problem 3 (Mapping): The mapping problem is to solve the allocation of accelerators for μops (the solution from Property 1), aiming to minimize the NoC communication costs during dataflow executions.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

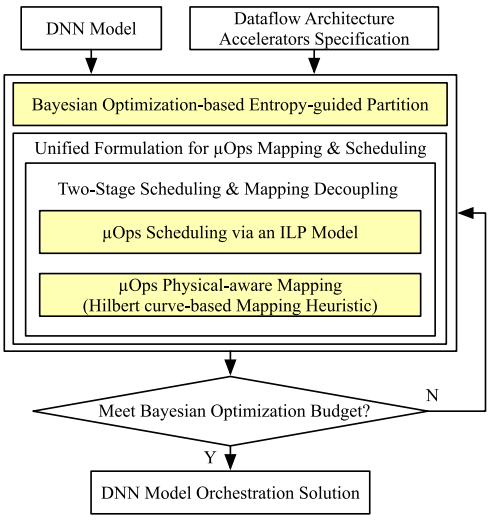


Fig. 5. Overview of Klotski v2 framework.

The three problems are not independent. In this article, they are solved altogether in the Klostki v2 framework.

III. Klotski v2

This section details Klotski v2. We first present an overview (Section III-A). Then, we introduce the partition strategy for different DNN models (Section III-B) and BO-based partition algorithms (Section III-C). Finally, we illustrate the unified scheduling and mapping formulation (Section III-D), plus the corresponding decoupled form (Section III-E).

A. Overview of Klotski v2

Fig. 5 is an overview of Klotski v2. The inputs of Klotski v2 are a DNN model and specifications of dataflow architecture accelerators. The specifications include hardware configurations for dataflow architecture accelerators, such as the NoC topology, routing algorithm, memory capacities, etc. The DNN model is partitioned to many μops by our proposed BO-based entropy-guided algorithm; via decoupled scheduling and mapping, Klotski v2 outputs a DNN model orchestration solution. A Hilbert curve-based mapping can be alternatively used in the mapping stage to further improve the algorithm runtime.

B. DNN Model Partition

The DNN model partition produces different μops with μtensors. μOp is a partial computation of a DNN operator, and μtensors are operands associated with the μop. Specifically, μtensors could be parts of activations, layer weight submatrices, etc. Unlike previous partition strategies, e.g., regular partition layerwise [27], a "partition-n-reduce" fashion [28], or homogeneous "*r*operator" generation [29], our DNN model partition is more fine-grained. Such finer granularity allows higher execution parallelism in dataflow architecture accelerators. Next, we introduce two examples of the proposed partition strategy for DNN models with convolutions and multihead attentions, separately.

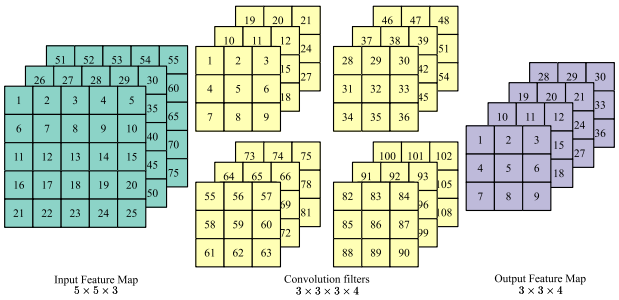


Fig. 6. In the example of a convolution, each element of input feature maps, filters, and output feature maps are numbered to distinguish between each other.

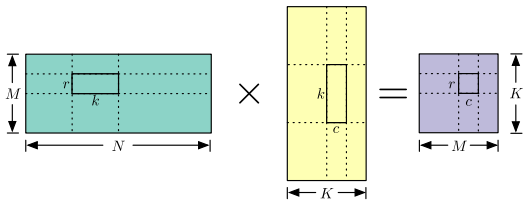


Fig. 7. Example of a GEMM operation with $M \times N$ and $N \times K$ matrices. The partition of the GEMM operation is parameterized by $\boldsymbol{s}(r, k, c)$.

1) Partition of Convolution: Assume the convolution computation is associated with a tuple (R, S, P, Q, C, K), where the tuple elements denote filter width and height, output width and height, input channel, and output channel, respectively. We partition the convolution with a four-element tuple $\boldsymbol{s}(h, w, ic, oc)$, where h, w, and oc refer to the output μtensors' height, width, and output channels, and ic is for the input channels.

Fig. 6 is an example of a convolution with (R, S, P, Q, C, K) equal to $(3, 3, 3, 3, 3, 4)$. The element of each tensor is numbered for μop and μtensor construction. If we utilize $\boldsymbol{s}(h, w, ic, oc) = (2, 2, 2, 3)$ to partition the convolution, we can produce 16 different μops. Each μop is a convolution with various subsets of original feature maps and filters. Table I summarizes these μops and associated μtensors. Numbers highlight how μtensors are formed from the original tensors. As Table I illustrates, if the dimension of output feature maps cannot be divisible by a corresponding element of $\boldsymbol{s}$, we treat the remainder as extra μops.

2) Partition of Multihead Attention: Multihead attention is the core computational structure of the Transformer [30]. The formulation of multihead attention is illustrated in

$$\text{Multihead}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{concat}(\boldsymbol{H}_1, \boldsymbol{H}_2, \ldots, \boldsymbol{H}_h) \tag{3}$$

where $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are the *query*, *key*, and *value* matrices. $\boldsymbol{H}_i, i \in \{1, 2, \ldots, h\}$ is the output of a scaled dot-product attention head as formulated in

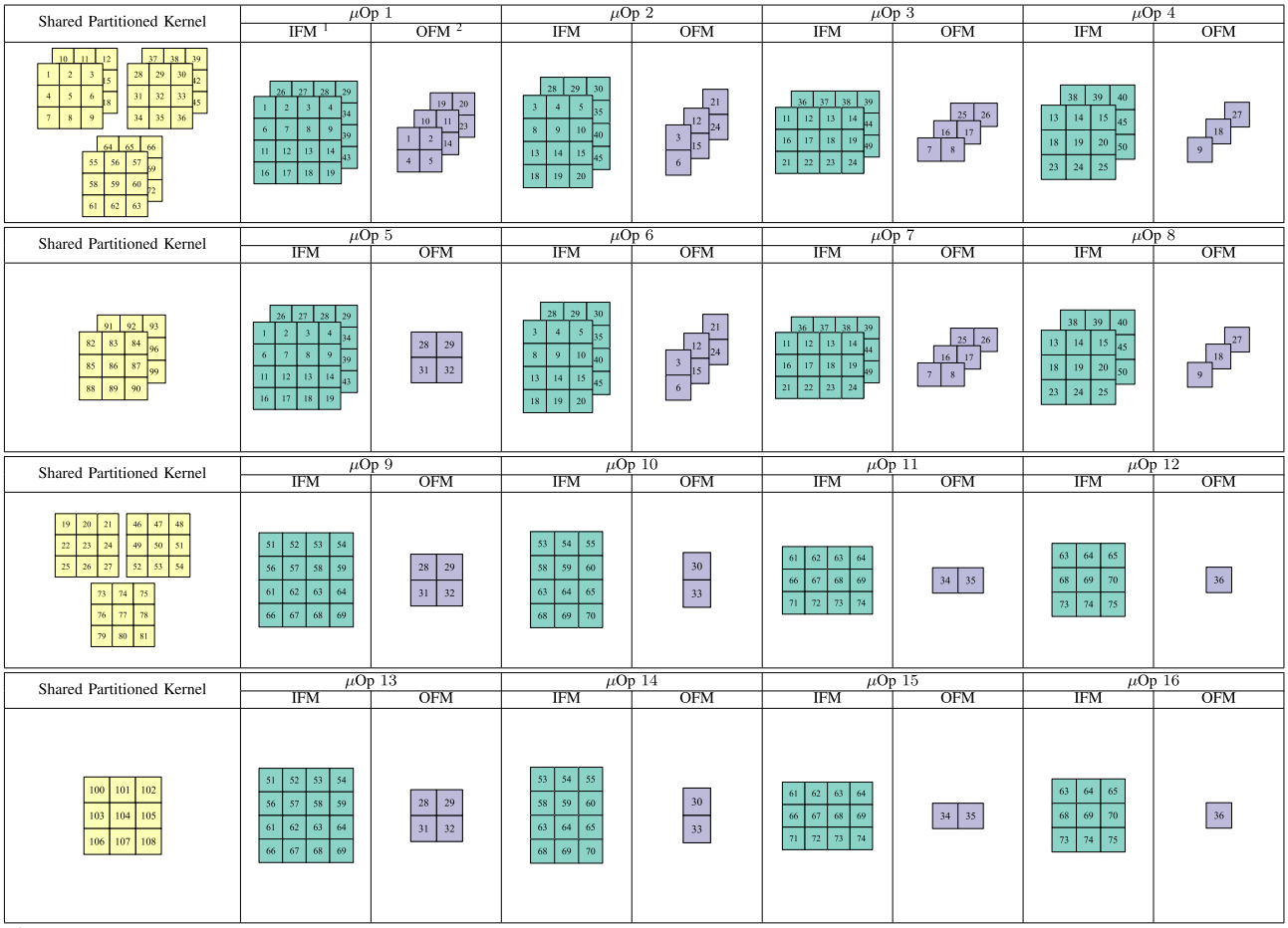
$$\boldsymbol{H}_i = \text{softmax}\left[\frac{\boldsymbol{Q}_i \boldsymbol{K}_i^\top}{\sqrt{d_k}}\right] \boldsymbol{V}_i \tag{4}$$

where $\boldsymbol{Q}_i$, $\boldsymbol{K}_i$, and $\boldsymbol{V}_i$ are the ith head of $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$. d_k is a constant.

Since the core of the multihead attention is a general matrix multiply (GEMM) operation, we partition the computation using a three-element tuple $\boldsymbol{s}(r, k, c)$, where r and c denotes the

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF 16 μOPS FOR THE PARTITION OF THE CONVOLUTION SHOWN IN FIG. 6



[1] Input feature map
[2] Output feature map

output μtensors' number of rows and columns, and k is a factor to partition the reduced dimension of GEMM. Fig. 7 details the meaning of $\boldsymbol{s}(r, k, c)$. Different partition strategies exist and could be more fine-granular than ours. We adopt $\boldsymbol{s}(r, k, c)$ for the tradeoff between the difficulty of implementation and relatively high execution efficiency for multihead attention.

C. Bayesian Optimization-Based Entropy-Guided Partition

What is the optimal DNN model partition strategy? In other words, what is the optimal $\boldsymbol{s}(h, w, ic, oc)$ or $\boldsymbol{s}(r, k, c)$ for each layer of convolution neural networks or Transformer models, respectively? Due to the unclear form between the generated μops and the execution performance of dataflow architecture accelerators, we formulate the problem as a design space exploration (DSE) and solve it with a BO-based method.

BO is an iterative trial-and-error optimization algorithm. It finds parameter values $\boldsymbol{s}$ that can receive promising results via a *surrogate model* and an *acquisition function*. The surrogate model is built from the Gaussian process [31], characterizing the probabilistic relations between $\boldsymbol{s}$ and the objective function, and the acquisition function serves as a guide. Our partition algorithm is listed in Algorithm 1. We construct the design space $\mathbb{D}$ of $\boldsymbol{s}$. In each iteration, we partition a DNN model with a specific $\boldsymbol{s}$ sampled from $\mathbb{D}$ or suggested from our acquisition function, i.e., the upper confidence bound (UCB) (lines 3 and 7). The produced μops are scheduled, mapped, and executed on dataflow architecture accelerators, and we evaluate with the objective function (line 4), as shown in

$$E(\boldsymbol{s}) = -\left(\sum_{i \in V} \frac{l(i)}{l(V)} \ln \frac{l(i)}{l(V)}\right) / (\alpha \cdot \text{makespan}) \tag{5}$$

where $l(V) = \sum_{i \in V} l(i)$. The set S is aggregated and used to construct a Gaussian process model, expecting to find better $\boldsymbol{s}$ in the following iteration (lines 6 and 7). The algorithm terminates if a predetermined budget T is met.

Our partition algorithm achieves the two basic requirements. First, we permit the computation of a μop to fully utilize the processing elements (PEs) of a single DNN accelerator with the construction of the design space. Second, we make the execution latency of μops as close as possible with (5). As a result, we avoid load unbalancing, i.e., many μops wait for a "bad" μop with long execution latency. Besides, if the DNN model size is larger than the memory capacity of all accelerators, the model is partitioned until each part can be held in the accelerator on-chip memory.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 BO-Based Entropy-Guided Partition

Require: G : a DNN model. $\mathbb{D}$: the design space for $\boldsymbol{s}$. T: optimization budget.

$S = \emptyset$; Sample $\boldsymbol{s} \in \mathbb{D}$;
for $i = 1 \rightarrow T$ **do**
  Partition G with $\boldsymbol{s}$;
  Execute μops and evaluate $E(\boldsymbol{s})$; ▷ Equation (5)
  $S = S \cup \{(\boldsymbol{s}, E(\boldsymbol{s}))\}$;
  Construct a Gaussian process model M_i with S;
  $\boldsymbol{s}^* = argmax_{\boldsymbol{s}\in\mathbb{D}} \text{UCB}(\boldsymbol{s}, M_i(\boldsymbol{s}))$; $\boldsymbol{s} = \boldsymbol{s}^*$
end for
return Optimal $\boldsymbol{s}^*$ from S.

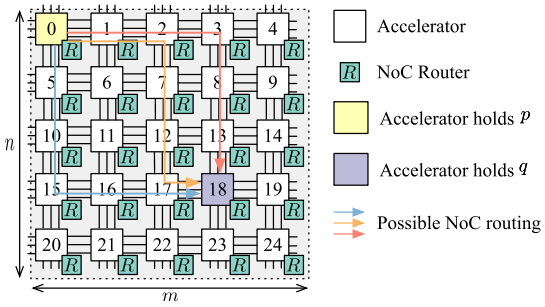


Fig. 8. Overview of an NoC communication.

D. Unified Formulation for μOps Scheduling and Mapping

We give a formal formulation for the scheduling and mapping with generated μops (Section III-C). Our formulation targets to minimize the makespan and reduce the NoC communication overhead.

We apply the list scheduling [32], [33], [34] to acquire the upper bound of the makespan for generated μops. The main idea is to schedule a ready μop, whose dependent producers are finished, from a predefined order for idle accelerators immediately. Theorem 1 gives the upper bound formally.

Theorem 1 (Upper Bound of the Makespan for μOps Scheduling): List scheduling achieves $2 - 1/|\boldsymbol{a}|$ times the optimal makespan for dataflow architecture accelerators.

With the upper bound (denote it as T), we can calculate the scheduling flexibility of each μop with ASAP and ALAP (Section II-B) accordingly. Suppose the earliest possible and the latest possible time slot for one μop i to issue to the PEs are S_i and L_i. We define a binary tensor $\boldsymbol{\mathcal{X}}$ with the size of $|V| \times T \times |\boldsymbol{a}|$ as the scheduling and mapping solution, shown in

$$\boldsymbol{\mathcal{X}}_{ijk} = \begin{cases} 1, & \mu\text{op } i \text{ is scheduled to the } k\text{th accelerator} \\ & \text{at the } j\text{th time slot} \\ 0, & \text{otherwise.} \end{cases} \tag{6}$$

A legal solution should satisfy following constraints.

μOp Constraint: A μop can only be scheduled within the scheduling flexibility (7). And it can only be issued to one individual accelerator

$$\sum_{k=1}^{|\boldsymbol{a}|}\sum_{j=1}^{T} \boldsymbol{\mathcal{X}}_{ijk} = 1, \ \sum_{k=1}^{|\boldsymbol{a}|}\sum_{j=S_i}^{L_i} \boldsymbol{\mathcal{X}}_{ijk} = 1 \quad \forall i \in V. \tag{7}$$

Each μop must be finished before T

$$\sum_{k=1}^{|\boldsymbol{a}|}\sum_{j=S_i}^{L_i} (j + l(i))\boldsymbol{\mathcal{X}}_{ijk} \le T \quad \forall i \in V \tag{8}$$

where $l(i)$ is the execution latency for μop i. The execution latency includes cycles needed to load operands from main memory or via NoC communication.

Constraint by Property 1: Property 1 determines the scheduling priority for μops to guarantee correct computations

$$\sum_{k=1}^{|\boldsymbol{a}|}\sum_{j=S_p}^{L_p} j \cdot \boldsymbol{\mathcal{X}}_{pjk} - \sum_{k=1}^{|\boldsymbol{a}|}\sum_{j=S_q}^{L_q} j \cdot \boldsymbol{\mathcal{X}}_{qjk} \le -(l(p)) \quad \forall p \ \ \forall q \in V \text{ and } p \prec q. \tag{9}$$

Computing Resource Constraint: In each time slot, the number of μops to be issued should be fewer than the number of idle (available) accelerators

$$\sum_{k=1}^{|\boldsymbol{a}|}\sum_{p=0}^{l(i)}\sum_{i=1}^{|V|} \mathcal{X}_{i(j-p)k} \le |\boldsymbol{a}| \quad \forall j = \{1, 2, \ldots, T\}. \tag{10}$$

Our target is to minimize the makespan T while alleviating the NoC communication overhead. The reasons to optimize the NoC communication overhead are twofold. First, NoC is responsible not only for data communication but also for MIMD coordination. Second, many μops result in data movement between accelerators. Our NoC formulation overhead is physical-aware, and we elucidate with an example shown in Fig. 8. The NoC is a mesh topology with an XY-YX routing algorithm [35]. The producer μop p is mapped to the accelerator 0, while the consumer μop q is in accelerator 18. m and n are the numbers of individual accelerators per row and column. The route path is not unique, but we can formulate the communication cost with

$$c_{p\prec q} = \text{Vol}(p) \cdot (|x_1 - x_2| + |y_1 - y_2|) \tag{11}$$

where p's assigned accelerator is at (x_1, y_1), and q's accelerator is at (x_2, y_2). For instance, the coordinate of the accelerator 22 is $(4, 2)$. Vol(p) is the data size of p's producers for the transfer. We have following equations to compute the locations of accelerators:

$$x_1 = \lfloor * \rfloor \frac{p}{m}, y_1 = p \bmod m \tag{12}$$

$$x_2 = \lfloor * \rfloor \frac{q}{m}, y_2 = q \bmod m \tag{13}$$

$$p - q \neq 0 \text{ if } \sum_{k=1}^{|\boldsymbol{a}|} \boldsymbol{\mathcal{X}}_{pjk} + \sum_{k=1}^{|\boldsymbol{a}|} \boldsymbol{\mathcal{X}}_{qjk} > 1, \ j \in \{1, 2, \ldots, T\} \tag{14}$$

$$p = \sum_{k=1}^{|\boldsymbol{a}|}\sum_{j=S_p}^{L_p} k\boldsymbol{\mathcal{X}}_{pjk}, q = \sum_{k=1}^{|\boldsymbol{a}|}\sum_{j=S_q}^{L_q} k\boldsymbol{\mathcal{X}}_{qjk}, \ p, q \in V. \tag{15}$$

Equations (12) and (13) are formulations of source and target accelerators' locations ((x_1, y_1) and (x_2, y_2)), respectively. Equation (14) prevents duplicated mappings if p and q are fired at the same time slot. Equation (15) decides which accelerator to map a μop. The entire NoC communication costs are the

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

summation of the point-to-point μtensors transmissions (11) between a producer and a consumer, as shown in

$$C = \sum_{p,q \in V} c_{p \prec q}. \tag{16}$$

The complete unified formulation for the scheduling and mapping is shown below

$$\begin{aligned} \underset{\boldsymbol{\mathcal{X}}}{\text{argmin}} \quad & T + \beta C \\ \text{s.t.} \quad & (7)-(15) \end{aligned} \tag{17}$$

where β is a coefficient to tradeoff T and C. And the length of a single time slot is determined by min $l(i) \quad \forall i \in V$. Nevertheless, two difficulties restrict us from solving (17). First, when the problem size becomes large, it costs high runtime to construct constraints like (10). Second, the problem cannot be solved with mathematical programming due to nonlinearity in (11)–(13). We introduce a two-stage methodology to decouple the scheduling and mapping, making the solution feasible contrapuntally.

E. Two-Stage Scheduling and Mapping Decoupling

We decouple the unified formulation (Section III-D) by defining two new variables. We define a binary matrix $\boldsymbol{X}$ with the size of $|V| \times T$ as the scheduling solution, as shown in

$$\boldsymbol{X}_{ij} = \begin{cases} 1, & \mu\text{op } i \text{ is scheduled to the } j\text{th time slot} \\ 0, & \text{otherwise.} \end{cases} \tag{18}$$

Define a $|V| \times |\boldsymbol{a}|$ binary matrix $\boldsymbol{Y}$ shown in (19) as the mapping solution

$$\boldsymbol{Y}_{ij} = \begin{cases} 1, & \mu\text{op } i \text{ is mapped to the } j\text{th accelerator} \\ 0, & \text{otherwise.} \end{cases} \tag{19}$$

As a result, (17) is transformed into two subproblems.

1) Subproblem of Scheduling: The formulation for the subproblem of scheduling is shown in

$$\begin{aligned} \underset{\boldsymbol{X}}{\text{argmin}} \quad & T \\ \text{s.t.} \quad & \sum_{j=S_i}^{L_i} \boldsymbol{X}_{ij} = 1, \sum_{j=S_i}^{L_i} (j + l(i)) \boldsymbol{X}_{ij} \le T \\ & \sum_{j=S_i}^{L_i} j\boldsymbol{X}_{ij} - \sum_{j=S_k}^{L_k} j\boldsymbol{X}_{kj} \le -(l(i)), \quad i \prec j \\ & \sum_{d=0}^{l(i)} \sum_{i=1}^{|V|} \boldsymbol{X}_{i(j-d)} \le |\boldsymbol{a}|, \quad j - d \ge 1, \quad 1 \le j \le T \end{aligned} \tag{20}$$

where the final constraint of (20) rephrases the computing resource constraint for (10). Such a constraint still requires a high runtime to construct. So, it can be relaxed, as shown in

$$\sum_{i \in I = \{i | j \in [S_i, L_i]\}} \boldsymbol{X}_{ij} \le |\boldsymbol{a}| \quad \forall j \in \{1, 2, \ldots, T\}. \tag{21}$$

We accommodate (14) only for pair of μops that can be parallelized during executions. The parallelism comes from inter-μops and intra-μops.

2) Subproblem of Mapping: Concerning the nonlinearity in (17), we transform the mapping problem to a mixed-integer linear programming (MLP), as shown below

$$\underset{\boldsymbol{Y}_p, \boldsymbol{Y}_q}{\text{argmin}} \quad k_1 + k_2 + n_1 + n_2 \tag{22}$$

$$\text{s.t.} \quad x_1 - x_2 = k_1 - k_2, \quad p - q = n_1 - n_2 \tag{23}$$

$$\frac{p}{m} - \epsilon \le x_1 \le \frac{p}{m}, \quad \frac{q}{m} - \epsilon \le x_2 \le \frac{q}{m} \tag{24}$$

$$p = a \cdot m + y_1, \quad q = b \cdot m + y_2 \tag{25}$$

$$0 \le y_1 \le m - 1, \quad 0 \le y_2 \le m - 1 \tag{26}$$

$$\delta - (1 - z) \cdot M \le p - q \le -\delta + Mz \tag{27}$$

$$M = |\boldsymbol{a}| + 1 \tag{28}$$

$$p = \sum_{k=1}^{|\boldsymbol{a}|} k\boldsymbol{Y}_{pk}, \quad q = \sum_{k=1}^{|\boldsymbol{a}|} k\boldsymbol{Y}_{qk} \tag{29}$$

$$k_1, k_2, n_1, n_2 \ge 0, \quad a, b, x_1, x_2 \in \mathbb{Z}, \quad z \in \{0, 1\}. \tag{30}$$

Take an example from (12) to (15). With newly incorporated six rational variables (k_1, k_2, n_1, n_2, y_1, and y_2), four integer variables (x_1, x_2, a, and b), and a binary variable z, we make the communication formulation between μop p and q solvable [from (22) to (30)]. In (26), the relation is constructed based on the condition: $\boldsymbol{X}_{pk} + \boldsymbol{X}_{qj} > 1, \ j \in \{1, 2, \ldots, T\}$ [see (14)]. In (24) and (27), ϵ and δ are two small constants. Specifically, we choose $\epsilon = 0.999$ and $\delta = 0.001$. p and q in (29) are for μop ID. Newly introduced variables like k_1, k_2, n_1, etc., are leveraged to linearize (16). Take minimizing $|x_1 - x_2|$ as an example, it can be linearized by two new variables k_1 and k_2, as shown in

$$\begin{aligned} \min \quad & k_1 + k_2 \\ \text{s.t.} \quad & x_1 - x_2 = k_1 - k_2 \\ & k_1, k_2 > 0. \end{aligned} \tag{31}$$

Equations (24), (25), and (26) are used to linearize (12) and (13).

Moreover, we enhance the modeling of the NoC communication cost based on (11), as shown in (32)

$$c_{p \prec q} = \frac{f}{\text{bw}} \cdot (|x_1 - x_2| + |y_1 - y_2|) + \frac{(\text{Vol}(u_p)/f - 1)f}{\text{bw}} \tag{32}$$

where f is the flit size and bw is the NoC link bandwidth. Equation (32) considers NoC communication in a pipeline fashion, and minimizing (22) equals minimizing $c_{p \prec q}$ (32).

A similar idea in (32) is also used to handle the latency of main memory access. The main memory access latency is obtained from the transmitted data volume size and main memory bandwidth accordingly. The division of transmitted data volume size and required data size decides the number of main memory accesses an accelerator needs.

IV. Hilbert Curve-Based Mapping Heuristic

Although the new two-stage methodology (Section III-E) enhances the formal mapping formulation, the runtime for solving these ILP problems is high, especially for the mapping subproblem, making the formulation less scalable with the problem size. We propose a Hilbert curve-based mapping heuristic to accelerate the solution. The heuristic, applied to

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

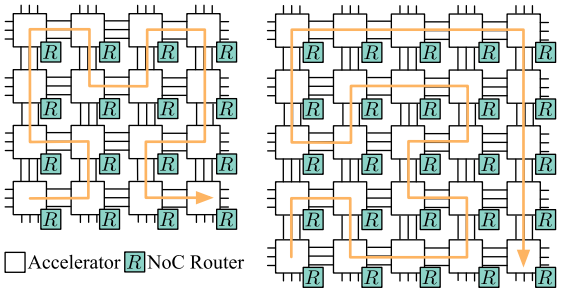


Fig. 9. Overview of Hilbert curve-based mapping heuristic. The line highlighted in orange shows the complete or a part of a Hilbert curve.

the subproblem of mapping, is a tradeoff between the solution quality and efficiency.

The main idea of the heuristic is to map μops to accelerators in a Hilbert curve manner, as shown in Fig. 9. Hilbert curve is a kind of fractal space-filling curve. A space-filling curve is a curve whose range reaches every point in a higher dimensional region [36]. For example, the Peano curve is the first proposed space-filling curve, admitting a surjective mapping from $\mathbb{R} \to \mathbb{R}^2$. It is formulated by $f : [0, 1] \to [0, 1]^2$, where $f(x) = \lim_{n\to\infty} f_n(x)$ [37]. The analytical formulation of the Peano curve using the parametric equation $t \to (\phi(t), \psi(t))$ is shown in (33) [37]

$$\phi(t) = \sum_{n=1}^{\infty} \frac{f_{2n-1}(t)}{3^n}, \quad \psi(t) = \sum_{n=1}^{\infty} \frac{f_{2n}(t)}{3^n}. \tag{33}$$

Hilbert curve $f(x)$ is a variant of the Peano curve, which has a good locality-preserving property among all the space-filling curves [38], [39]. The locality-preserving property explicitly illustrates that two points close to each other in 1-D space are also close after folding. Compared to zig-zag mapping, Hilbert curve-based mapping can exploit more locality by tightly folding NoC communications to avoid long transmission. Using the quaternary Cantor set, the analytical expression of the Hilbert curve can be described as follows [37]:

$$f(x) = \sum_{j=1}^{\infty} \frac{1}{2^j} (-1)^{e_{0j}} \text{sign}(q_j) \begin{pmatrix} (1-d_j)q_j - 1 \\ 1 - d_j q_j \end{pmatrix} \tag{34}$$

where e_{kj} is the number of k's preceding q_j $(\text{mod}\,2)$ and $d_j = e_{0j} + e_{3j}$ $(\text{mod}\,2)$. Consequently, we design a series of rules to map μops based on the property.

Assuming two μops i and j, we define the following rules in the mapping based on the Hilbert curve.

Rule 1: Map j to the accelerator where i is mapped if $i \prec j$.

Rule 2: μOps without producer-consumer relations ($i \not\prec j$ and $j \not\prec i$) are mapped *w.r.t.* a Hilbert curve.

If the topology of accelerators does not admit the existence of a Hilbert curve, we adopt part of a Hilbert curve (e.g., the 5×5 topology applies a part of a Hilbert curve with the order of 3 in Fig. 9 [40]).

The rationale for the proposed heuristic lies in the interpretation of (27) and (32). Equation (32) aims to minimize the NoC communication cost while (27) maximizes the accelerator utilization. Minimizing the NoC communication cost equals mapping producer μops to consumer μops as closely as possible (rule 1). Maximizing the accelerator utilization requires scattering μops that can be parallelized as much as possible (rule 2). The two counteracting forces demand a mapping method to investigate the locality between μops. Hilbert curve can preserve locality when we map μops to underlying 2-D accelerator arrays [41]. We use breadth-first search to decide whether to apply rule 2 for μops.

Klotski v2's result is a vector of pairs. Each pair is (μop id, acc. id), where "μop id" refers to a μop ID, and "acc. id" denotes an accelerator's ID. The sequence of pairs denotes the scheduling precedence. According to the vector, each controller of an accelerator is assigned specific μops in a "first come, first served" (FCFS) fashion. These μops are scheduled and executed following the dataflow execution manner. For each accelerator, its controller snoops the readiness of assigned μops' operands and the availability of computing units. μop is fired immediately if required resources are ready. Computed results are sent to the consumer μops' locations, e.g., an accelerator or the main memory.

V. Experiments

This section details implementations (Section V-A), experimental settings (Section V-B), and evaluation results (Sections V-C–V-F).

A. Implementation Details

We build an in-house simulator for the dataflow architecture accelerators. The simulator is built up ahead of a real chip and used to conduct the theoretical analysis and the proof of the concept at the early chip design stage. The simulator leverages a proprietary NoC with the mesh topology to connect individual DNN accelerators. The NoC applies the XY-YX routing algorithm [35]. The flit size is 512 bytes, and each packet size is 64 bytes. Each individual DNN accelerator adopts KC-partition and equips with a 12×14 PE array. PEs are shared with an on-chip SRAM memory of 5.86 MB. The size of registers in each PE is 5 KB. The precision of computations are 32 bits. The bandwidth of an off-chip memory is set as 22.35 GB/s. The frequency is 1 GHz. MAESTRO [43] is applied in the simulator to model the execution performance of one individual DNN accelerator. Our in-house simulator is event-driven. It achieves a tradeoff between the engineering efforts and acceptable modeling accuracy to study the DNN orchestration. For example, the simulator permits no NoC deadlocks since our problem is decoupled from how to handle deadlocks. When NoC link contention comes into play, our event-driven mechanism is allowed to handle it by using a timeline model, i.e., each NoC link maintains a private timeline. An NoC data transmission can be achieved if we can find an available time slot from the private timeline. The time slot is then masked and hidden from the other NoC data transmission events. We assume the off-chip memory capacity is unlimited, and leverage a simplified analytical model to handle the main memory architecture and related manipulations. We do not make assumptions about how access to the main memory is performed, like whether the address is continuous or which memory bank's stored data is used. Our method is still applicable because we emphasize

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

TABLE II
EXPERIMENTAL RESULTS FOR THE 3 × 3 TOPOLOGY

Workload	Method	HUR [1]	Cycles	Ratio	Overall Runtime	Ratio
VGG16	Baseline 1	1.0000	1.2283E + 08	1.0000	−− [2]	−−
	Baseline 2	2.1196	5.5633E + 07	0.4529	472.6190	1.0000
	Baseline 3	2.5945	4.4677E + 07	0.3637	302.6349	0.6403
	Klotski [42]	2.8298	4.0659E + 07	0.3310	878.8832	1.8596
	Klotski v2	**3.4785**	**3.3021E+07**	**0.2688**	**127.2110**	**0.2692**
VGG19	Baseline 1	1.0000	1.5523E + 08	1.0000	−−	−−
	Baseline 2	1.9895	7.4207E + 07	0.4781	543.4454	1.0000
	Baseline 3	2.0059	7.1980E + 07	0.4637	654.6817	1.2047
	Klotski [42]	2.6001	5.5381E + 07	0.3568	887.5791	1.6332
	Klotski v2	**3.1381**	**4.5488E+07**	**0.2930**	**193.5358**	**0.3561**
ResNet50	Baseline 1	1.0000	7.7422E + 07	1.0000	−−	−−
	Baseline 2	1.6287	5.7060E + 07	0.7370	583.6488	1.0000
	Baseline 3	2.0377	5.2506E + 07	0.6782	693.7723	1.1887
	Klotski [42]	2.1171	4.8174E + 07	0.6222	1058.8011	1.8141
	Klotski v2	**2.3511**	**4.5159E+07**	**0.5833**	**147.9278**	**0.2535**
ResNet152	Baseline 1	1.0000	1.8984E + 08	1.0000	−−	−−
	Baseline 2	1.2147	1.7102E + 08	0.9009	867.0853	1.0000
	Baseline 3	1.2440	1.6623E + 08	0.8757	1004.5680	1.1586
	Klotski [42]	1.3704	1.5947E + 08	0.8400	2800.9154	3.2302
	Klotski v2	**1.4204**	**1.5753E+08**	**0.8298**	**394.4732**	**0.4549**
Inception	Baseline 1	1.0000	2.5122E + 07	1.0000	−−	−−
	Baseline 2	2.2822	1.6345E + 07	0.6506	483.8282	1.0000
	Baseline 3	3.2419	1.4369E + 07	0.5719	830.6549	1.7168
	Klotski [42]	3.5961	1.3348E + 07	0.5313	2393.7024	4.9474
	Klotski v2	**3.8385**	**1.2108E+07**	**0.4820**	**436.6441**	**0.9025**
Transformer v1	Baseline 1	1.0000	9.8680E + 07	1.0000	−−	−−
	Baseline 2	1.8003	4.9726E + 07	0.5039	406.7188	1.0000
	Baseline 3	1.8941	5.5706E + 07	0.5645	881.4852	2.1673
	Klotski [42]	2.8458	3.7076E + 07	0.3757	788.2354	1.9380
	Klotski v2	**2.9049**	**3.6321E+07**	**0.3681**	**206.3242**	**0.5073**
Transformer v2	Baseline 1	1.0000	2.8617E + 08	1.0000	−−	−−
	Baseline 2	1.5480	1.2102E + 08	0.4229	2948.2297	1.0000
	Baseline 3	1.5258	1.4057E + 08	0.4912	3828.6762	1.2986
	Klotski [42]	**2.4750**	**8.6659E+07**	**0.3028**	3138.7212	1.0646
	Klotski v2	2.4383	8.7966E + 07	0.3074	**2450.3780**	**0.8311**

[1] Hardware utilization ratio
[2] Not applicable

scheduling and mapping on the accelerator array executing in a dataflow fashion.

In Klotski v2, we leverage an open-source tool *nn_dataflow* [44] as the front end of DNN models. The DNN model partition is implemented based on the framework. In the partition algorithm, the maximal budget of the BO-based entropy-guided partition is set as 50. The budget is set based on algorithm runtime and solution quality tradeoffs. We use Gurobi v10.0 [45] as the ILP or MLP solver in the two-stage methodology. While Klotski uses the decoupled mapping form [42], Klotski v2 adopts the Hilbert curve-based mapping heuristic (Section IV). We implement the pure algorithm of Klotski v2 with 5K lines of Python codes, including partition, scheduling, and mapping.

The experiments are conducted with our high-performance server equipping 80 Intel Xeon CPU E7-4830 v2 cores with a 1 TB main memory. In the evaluation, we adopt three kinds of scales of dataflow architecture accelerators, in which the accelerators are organized in 3 × 3, 4 × 4, and 5 × 5 arrays, i.e., we have 9, 16, and 25 accelerators in total. Such hardware scaling is used to denote different problem sizes, allowing us to fully evaluate the methods' scalabilities. It also provides us with the performance scaling of dataflow architecture accelerators.

B. Baselines and Wokrloads

Our baselines are constructed based on Tangram [11], the atomic dataflow [22], and the SET [23], [24]. Tangram schedules a DNN model per layer and proposes a zig-zag mapping strategy [11]. The atomic dataflow combines with simulated annealing in partition and dynamic programming in scheduling [22]. SET directly applies simulated annealing for scheduling and mapping with a proposed resource allocation tree (RA Tree) data structure [23]. These are representative solutions to the problem in traditional scalable DNN accelerators. However, since our problem originates from a new architecture, we cannot directly compare Klotski v2 with them. We have made appropriate modifications to Tangram [11] and the atomic dataflow [22] to enable these methods to orchestrate a DNN workload with our architecture model. And we also construct a baseline from the inspiration of SET [23]. For Tangram [11], we schedule a DNN model layer-wise and allocate accelerators following the zig-zag manner, and we term it as "baseline 1." The atomic dataflow schedules a workload per round with heuristics due to the synchronization in traditional scalable DNN accelerators [22]. We remove the synchronization and consider the heuristics proposed by the atomic dataflow for all ready μops rather than a candidate set in each round [22]. We term the modified

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EXPERIMENTAL RESULTS FOR THE 4 × 4 TOPOLOGY

Workload	Method	HUR	Cycles	Ratio	Overall Runtime	Ratio
VGG16	Baseline 1	1.0000	1.2283E + 08	1.0000	−−	−−
	Baseline 2	2.5617	4.5869E + 07	0.3734	474.3824	1.0000
	Baseline 3	1.4475	4.4491E + 07	0.3622	667.8275	1.4130
	Klotski [42]	3.8387	3.0670E + 07	0.2497	1147.8127	2.4196
	Klotski v2	**3.9954**	**2.8655E+07**	**0.2333**	**170.3615**	**0.3591**
VGG19	Baseline 1	1.0000	1.5523E + 08	1.0000	−−	−−
	Baseline 2	2.5229	5.8049E + 07	0.3740	568.8429	1.0000
	Baseline 3	2.8293	5.1389E + 07	0.3311	**511.0257**	**0.8984**
	Klotski [42]	3.2935	4.3795E + 07	0.2821	1473.2602	2.5899
	Klotski v2	**3.3438**	**4.3203E+07**	**0.2783**	532.3652	0.9359
ResNet50	Baseline 1	1.0000	7.7422E + 07	1.0000	−−	−−
	Baseline 2	1.7355	5.3365E + 07	0.6893	541.8091	1.0000
	Baseline 3	2.0600	5.1700E + 07	0.6678	453.1319	0.8363
	Klotski [42]	2.3061	4.6260E + 07	0.5975	1019.2198	1.8811
	Klotski v2	**3.3295**	**4.3955E+07**	**0.5677**	**171.8228**	**0.3171**
ResNet152	Baseline 1	1.0000	1.8984E + 08	1.0000	−−	−−
	Baseline 2	1.2523	1.6578E + 08	0.8733	793.7304	1.0000
	Baseline 3	1.3900	1.6150E + 08	0.8507	1008.1550	1.2701
	Klotski [42]	1.3878	1.5754E + 08	0.8299	1953.6979	2.4614
	Klotski v2	**1.4340**	**1.5603E+08**	**0.8219**	**377.1654**	**0.4752**
Inception	Baseline 1	1.0000	2.5188E + 07	1.0000	−−	−−
	Baseline 2	2.5323	1.5183E + 07	0.6028	**419.3479**	**1.0000**
	Baseline 3	3.8637	1.2331E + 07	0.4895	690.2394	1.6460
	Klotski [42]	4.4312	1.0781E + 07	0.4280	1442.6460	3.4402
	Klotski v2	**4.7490**	**9.8039E+06**	**0.3892**	450.9182	1.0753
Transformer v1	Baseline 1	1.0000	7.5728E + 07	1.0000	−−	−−
	Baseline 2	2.1551	3.1879E + 07	0.4210	**396.3333**	**1.0000**
	Baseline 3	1.9504	4.1515E + 07	0.5482	982.5255	2.4790
	Klotski [42]	2.9445	2.7499E + 07	0.3631	8387.1895	21.1620
	Klotski v2	**3.1058**	**2.6071E+07**	**0.3443**	1209.3937	3.0515
Transformer v2	Baseline 1	1.0000	2.2122E + 08	1.0000	−−	−−
	Baseline 2	1.9136	7.5683E + 07	0.3421	2952.2818	1.0000
	Baseline 3	1.6918	9.8007E + 07	0.4430	3236.4233	1.0962
	Klotski [42]	**2.7293**	**6.0749E+07**	**0.2746**	3125.2526	1.0586
	Klotski v2	2.6726	6.2038E + 07	0.2804	**2256.7639**	**0.7644**

method as "baseline 2." We also build up "baseline 3" from the inspiration of SET [23]. SET focuses on layer-wise granularity and due to uncertainty in the T Cut modeling in our dataflow execution model, we cannot directly apply RA Tree [23]. So, we only implement simulated annealing to optimize the scheduling and mapping simultaneously. The partition strategy for baseline 3 is employed from Klotski v2's adopted partition, which facilitates the comparison between simulated annealing and Klotski or Klotski v2 in scheduling and mapping. We design four different mutation options for simulated annealing in baseline 3, involving switching a number of pairs of μops scheduling precedences or changing μops' mapped accelerators. Our modifications adhere to Tangram and atomic dataflow's original ideas but extend their applicability to the dataflow architecture accelerators [11], [22].

Our workloads include both traditional convolutional and Transformer-based neural networks. The CNNs are VGG16, VGG19 [46], ResNet50, ResNet152 [47], and Inception v3 [48]. The Transformer-based neural networks are three cascaded attention layers from BERT [49] and GPT3 [2]. We term these two workloads as Transformers v1 and v2. In particular, the number of heads, the dimensionality of outputs, and the hidden dimension of the specified inner linear transformation layer are 12, 768, and 3072 for Transformer v1 [49]. The corresponding parameters for Transformer v2 are 16, 1536, and 6144 [2]. We stack three attention layers instead of using the entire BERT and GPT3. We mainly focus on running cycles as our performance metric. This is because running cycles are the first available and critical metric for studying the new architecture, which our simulator supports.

C. Comparison of Methodologies for 3 × 3 Topology

Table II lists related results for 3×3 topology. In particular, the overall runtime also includes Gurobi solving runtime. Baseline 1 does not partition a DNN model and straightly schedule and map the model layer-wise. The entire procedure is deterministic, and no search or tuning is incorporated. Hence, the runtime results for baseline 1 are not applicable. Baseline 2 is also deterministic except for the employed simulated annealing for partition. In the experiments, we set the running budgets for the simulated annealing in baseline 2 with 50 to align with the settings of Klotski and Klotski v2. And we set the budgets of the simulated annealing for baseline 3 as 100 to tradeoff the cycle results and overall runtime.

Klotski is better than baseline 1 by 53.86%, baseline 2 by 19.13%, and baseline 3 by 19.28% in average performance across all workloads. Klotski v2 achieves 56.29%, 23.38%, and 23.52% fewer running cycles compared to three baselines. Compared to Klotski, Klotski v2 achieves 5.26% better performance than Klotski. The Hilbert curve-based mapping heuristic can lead to a solution with a higher hardware

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
EXPERIMENTAL RESULTS FOR THE 5 × 5 TOPOLOGY

Workload	Method	HUR	Cycles	Ratio	Overall Runtime	Ratio
VGG16	Baseline 1	1.0000	1.2283E + 08	1.0000	−−	−−
	Baseline 2	2.7157	4.2621E + 07	0.3470	466.9748	1.0000
	Baseline 3	2.7067	4.3373E + 07	0.3531	844.4347	1.8083
	Klotski [42]	**4.7819**	**2.4240E+07**	**0.1973**	1640.0338	3.5120
	Klotski v2	4.5831	2.5242E + 07	0.2055	**188.1977**	**0.4030**
VGG19	Baseline 1	1.0000	1.5523E + 08	1.0000	−−	−−
	Baseline 2	2.8346	5.0412E + 07	0.3248	569.8779	1.0000
	Baseline 3	3.1592	4.5732E + 07	0.2946	547.5052	0.9607
	Klotski [42]	3.6598	3.9046E + 07	0.2515	2755.4077	4.8351
	Klotski v2	**3.9674**	**3.6264E+07**	**0.2336**	**225.3845**	**0.3955**
ResNet50	Baseline 1	1.0000	7.7422E + 07	1.0000	−−	−−
	Baseline 2	1.8228	5.0868E + 07	0.6570	560.7615	1.0000
	Baseline 3	2.0233	4.5416E + 07	0.5866	881.2536	1.5715
	Klotski [42]	2.0870	4.4029E + 07	0.5687	1672.0000	2.9817
	Klotski v2	**2.5675**	**4.1409E+07**	**0.5348**	**237.4416**	**0.4234**
ResNet152	Baseline 1	1.0000	1.8984E + 08	1.0000	−−	−−
	Baseline 2	1.2575	1.6460E + 08	0.8671	858.0045	1.0000
	Baseline 3	1.3157	1.5697E + 08	0.8269	1024.4990	1.1940
	Klotski [42]	1.3552	**1.5240E+08**	**0.8028**	4505.7838	5.2515
	Klotski v2	**1.4466**	1.5467E + 08	0.8148	**379.1770**	**0.4419**
Inception	Baseline 1	1.0000	2.5180E + 07	1.0000	−−	−−
	Baseline 2	2.8642	1.2733E + 07	0.5057	514.9384	1.0000
	Baseline 3	3.4672	1.0344E + 07	0.4108	815.6243	1.5839
	Klotski [42]	4.3158	**8.3088E+06**	**0.3300**	2787.1383	5.4126
	Klotski v2	**4.5160**	8.8515E + 06	0.3515	**417.0694**	**0.8099**
Transformer v1	Baseline 1	1.0000	7.5900E + 07	1.0000	−−	−−
	Baseline 2	2.8797	2.3911E + 07	0.3150	**400.9067**	**1.0000**
	Baseline 3	2.2687	3.5771E + 07	0.4713	1065.8944	2.6894
	Klotski [42]	**4.4841**	**1.8098E+07**	**0.2384**	4283.2294	10.8071
	Klotski v2	4.2296	1.9187E + 07	0.2528	1760.9397	4.4431
Transformer v2	Baseline 1	1.0000	2.2186E + 08 [1]	1.0000	−−	−−
	Baseline 2	2.1833	5.1316E + 07	0.2313	2905.5202	1.0000
	Baseline 3	2.0565	8.0859E + 07	0.3645	3124.9409	1.0755
	Klotski [42]	**3.9652**	**4.1936E+07**	**0.1890**	13700.3761	4.7153
	Klotski v2	3.7145	4.4766E + 07	0.2018	**1366.3860**	**0.4703**

[1] The on-chip buffer size is increased to execute Transformer v2.

utilization ratio for a small scale of the topology. Thus, Klotski v2 leads to better performance. On the other hand, the results disclose that more considerations are needed to distinguish which set of μops can be parallelized in the formal mapping formulation (14) or (27). It is noting that the performance benefits received from Klotski are accompanied by a very high solution runtime. Klotski needs 89.46% or 45.76% more time to schedule and map μops than baselines 2 and 3. However, Klotski v2 improves the overall runtime by 66.88%, attaining a better tradeoff between the solution quality and solving runtime.

For CNN workloads, Klotski outperforms baselines 1–3 by an average of 44.42%, 15.29%, and 9.36%in cycles. And Klotski v2 achieves a higher ratio, i.e., 48.58%, 21.63%, and 16.14%, respectively. Klotski v2 obtains an average of 55.68% and 28.01% better hardware utilization ratio than baselines 2 and 3. Baseline 1 does not partition the workload, leading to a naturally low hardware utilization ratio if neural networks are not complicated.

For Transformer workloads (Transformers v1 and v2), the profits from performance improvement are relatively larger. Klotski achieves an average of 67.85%, 27.53%, and 36.96% fewer cycles, while Klotski v2 attains an average of 67.71%, 27.21%, and 36.68% higher performance than the three baselines. Nevertheless, Klotski costs more time to map Transformer workloads to accelerators. And Klotski v2 alleviates the long overall runtime by an average of 32.35% for these Transformer-like models.

D. Comparison of Methodologies for 4 × 4 Topology

The results of the 4 × 4 topology are shown in Table III. Klotski and Klotski v2 present continuous high-performance improvements compared to three baselines. On average, Klotski is 56.51%, 15.37%, and 18.14% better than baseline 1–3 for all workloads, including CNNs and two Transformer networks. For Klotski v2, the corresponding numbers are 57.37%, 17.06%, and 19.78%, respectively. Moreover, Klotski v2 achieves 6.97% higher performance and 72.13% better overall runtime than Klotski.

E. Comparison of Methodologies for 5 × 5 Topology

Table IV shows the results for 5 × 5 topologies. The performance benefits increased compared to experiments conducted in 3 × 3 and 4 × 4 topologies. Klotski attains an average of 62.22%, 17.25%, and 21.61% higher performance than baselines 1–3. And Klotski v2 receives 61.95%, 16.67%, and 21.05% improvements, respectively. The results indicate that baselines are not scalable enough when number of accelerators is increased. At the same time, the formal mapping formulation and proposed Hilbert curve-based mapping heuristic can handle the problem with a larger solution space. Besides, the

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

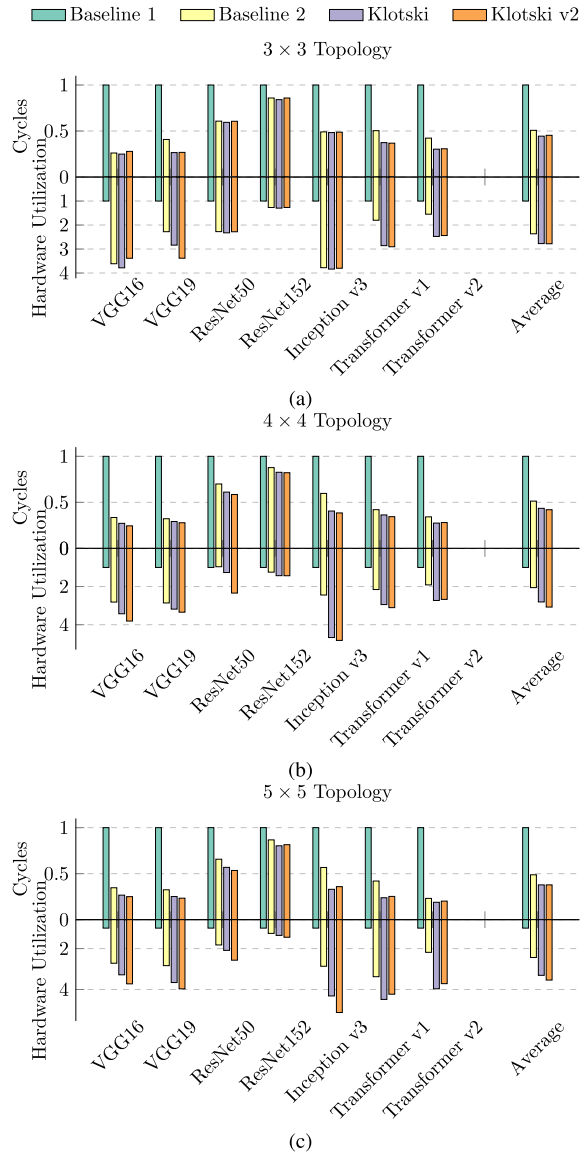


Fig. 10. Cycles and hardware utilization comparisons with different topologies. (a) 3×3 topology. (b) 4×4 topology. (c) 5×5 topology.

results also show the superiority of the Hilbert curve-based mapping heuristic over prior zig-zag mapping fashion. Our proposed mapping heuristic is a tradeoff between hardware utilization and producer-consumer reusability. In other words, scattering μops to accelerators makes each accelerator fully utilized as more as possible, while reusing producers for consumers emphasizes mapping these μops to the same accelerator as much as possible. The two antagonized objectives can be balanced under the Hilbert curve-based mapping heuristic.

It is worth noting that Klotski v2 achieves a comparable performance compared to Klotski in the 5×5 topology. Such degradation discloses that the Hilbert curve-based mapping heuristic cannot always receive higher performance benefits than the formal mapping formulation [42]. Compared to Klotski [42], Hilbert curve-based mapping heuristic is good at small scales of accelerators, while for larger problems, it achieves a marginal comparison. We also notice that for Klotski v2, although the hardware utilization ratio is slightly higher than that of Klotski, more running cycles are used to execute workloads like ResNet152 and Inception. The rationale lies in more NoC communication overhead with the Hilbert curve-based mapping heuristic in the larger topology scale, compared with Klotski. We apply part of Hilbert curve-based mapping for 3×3 and 5×5 topologies with two rules since these topologies do not admit the existence of a Hilbert curve (Section IV). In the 3×3 topology, an average of 3.86% more NoC communication overhead is caused for all DNN workloads. For 5×5 topology, by applying a part of the Hilbert curve-based mapping heuristic, we incur an average of 13.82% higher NoC communication cost while achieving comparable hardware utilization across all DNN workloads. It suggests the Hilbert curve-based mapping does not cause much loss in NoC communication for a small topology. While the heuristic can lead to higher NoC overhead on a larger scale. However, it achieves a tradeoff for mapping efficiency when the problem size becomes large, i.e., the algorithm runtime of Klotski v2 is greatly lessened. Klotski v2 outperforms Klotski by 85.41% in overall runtime. A significant contribution to the efficiency of Klotski v2 comes from mapping Transformer v2 to accelerators in 5×5 topology. The problem size is too big for Klotski to solve within an acceptable time.

We also find that baseline 3 achieves comparable running cycles with baseline 2 when the problem size is small (3×3 and 4×4 topology). Once the topology size is increased or the DNN model becomes quite large like Transformer models, baseline 3 does not scale well. It suggests that simulated annealing can achieve a good results for small problem but is not scalable for a larger problem.

F. Ablation Study

We investigate the effectiveness of scheduling and mapping by Klotski and Klotski v2 with an ablation study. This is achieved by applying Klotski, Klotski v2, and baseline 2 with the same μops and μtensors generated from a partition. The experimental settings allow us to solely study the scheduling and mapping with different methodologies. Baseline 1's results are also compared. Baseline 3 has been compared and analyzed in Sections V-C–V-E. So, we do not analyze baseline 3 in this section. All the results are listed in Fig. 10. In the 3×3 topology, Klotski outperforms baseline 1 by an average of 44.43% in cycles for all workloads. And this number is 45.35% for Klotski v2 [Fig. 10(a)]. Performance improvement is also observed in the results, as shown in Fig. 10(b) and (c). For example, in the 4×4 and 5×5 topology, Klotski v2 reduces the cycles by an average of 41.99% and 37.82% than baseline 1. And compared to baseline 2, Klotski v2 is 10.96% better. The comparison of the hardware utilization ratio provides the reason for the performance improvement. Generally, the higher the hardware utilization ratio, the higher the performance. Klotski and Klotski v2 achieve an average of $2.2\times \sim 3.5\times$ better in the hardware utilization ratio compared to baseline 1. And Klotski v2 receives slightly better than Klotski in these metrics for most workloads. The results demonstrate that Klotski v2 effectively improves the scheduling and mapping solution quality. Moreover, with the

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

BO-based entropy-guided partition, Klotski v2 can outperform baselines more. Besides, we also observe that when the number of accelerators is increased, i.e., the topology size becomes larger. Klotski and Klotski v2 can acquire a higher hardware utilization ratio, leading to higher performance improvement benefits for most workloads.

We summarize two lessons learned from experiments discussed in Sections V-C–V-E, and Fig. 10. *First, partitioning a DNN model into μops allows better execution performance, even for cascaded layers structures.* The claim can be concluded from comparisons between Klotski v2 and baseline 1 with different DNN models. Particularly, baseline 1 fails to explore opportunities to parallelize the cascaded layers executions (VGG16 and VGG19). *Second, the improvement of the execution performance is nonlinear to the increased hardware utilization ratio.* Compared to baseline 2, Klotski achieves higher the hardware utilization, as listed in Tables II–IV and Fig. 10. However, the gained hardware utilization ratio does not contribute to execution performance improvement expectedly. The reason stems from multiple factors. For example, different μops lead to various computation and memory access tradeoffs. Smaller sizes in μtensors do not represent higher performance efficiency.

VI. Conclusion

In this article, we propose Klotski v2, an improved DNN model orchestration framework for the newly emerged dataflow architecture accelerators. Four key techniques are proposed in Klotski v2.

1) BO-based entropy-guided partition generates μops for various DNN workloads, achieving load balancing and promising makespan from a global view.
2) A unified formulation for μops scheduling and mapping is characterized.
3) A two-stage methodology is proposed to decouple the scheduling and mapping, making the mathematical formulation solvable with off-the-shelf optimizers.
4) A Hilbert curve-based mapping heuristic enhances Klotski v2's solution efficiency and attains a new tradeoff between solution quality and algorithm runtime.

Experiments demonstrate that Klotski v2 can achieve an average of 21.57% higher execution performance improvement than the two baselines. With the Hilbert curve-based mapping heuristic, we improve the efficiency of Klotski by an average of 63.50% across different DNN workloads.

References

[1] OpenAI, "GPT-4 technical report," 2023, *arXiv:2303.08774*.
[2] T. Brown et al., "Language models are few-shot learners," in *Proc. NIPS*, vol. 33, 2020, pp. 1877–1901.
[3] R. Thoppilan et al., "LaMDA: Language models for dialog applications," 2022, *arXiv:2201.08239*.
[4] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with clip latents," 2022, *arXiv:2204.06125*.
[5] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. ISCA*, 2017, pp. 1–12.
[6] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. ISCA*, 2016, pp. 367–379.
[7] H. Genc et al., "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. DAC*, 2021, pp. 769–774.
[8] "NVIDIA NVDLA deep learning accelerator." 2017. [Online]. Available: http://nvdla.org
[9] D. Kim, J. Kung, S. Chai, S. Yalamanchili, and S. Mukhopadhyay, "Neurocube: A programmable digital Neuromorphic architecture with high-density 3D memory," in *Proc. ISCA*, 2016, pp. 380–392.
[10] S. Venkataramani et al., "SCALEDEEP: A scalable compute architecture for learning and evaluating deep networks," in *Proc. ISCA*, 2017, pp. 13–26.
[11] M. Gao, X. Yang, J. Pu, M. Horowitz, and C. Kozyrakis, "TANGRAM: Optimized coarse-grained dataflow for scalable NN accelerators," in *Proc. 24th ASPLOS*, 2019, pp. 807–820.
[12] D. Abts et al., "A software-defined tensor streaming multiprocessor for large-scale machine learning," in *Proc. ISCA*, 2022, pp. 567–580.
[13] N. P. Jouppi et al., "TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," *Proc. 50th ISCA*, 2023, pp. 1–14.
[14] J. Vasiljevic et al., "Compute substrate for software 2.0," *IEEE Micro*, vol. 41, no. 2, pp. 50–55, Mar./Apr. 2021.
[15] K. Sankaralingam et al., "The Mozart reuse exposed dataflow processor for AI and beyond," in *Proc. ISCA*, 2022, pp. 978–992.
[16] D. Ignjatović, D. W. Bailey, and L. Bajić, "The wormhole AI training processor," in *Proc. ISSCC*, vol. 65, 2022, pp. 356–358.
[17] R. Prabhakar et al., "SambaNova SN40L: Scaling the AI memory wall with dataflow and composition of experts," 2024, *arXiv:2405.07518*.
[18] J. B. Dennis and D. P. Misunas, "A preliminary architecture for a basic data-flow processor," in *Proc. ISCA*, 1974, pp. 126–132.
[19] J. B. Dennis, "Data flow supercomputers," *Computer*, vol. 13, no. 11, pp. 48–56, 1980.
[20] T. Nowatzki, V. Gangadhar, and K. Sankaralingam, "Heterogeneous Von Neumann/dataflow microprocessors," *Commun. ACM*, vol. 62, no. 6, pp. 83–91, 2019.
[21] Y. Shen, M. Ferdman, and P. Milder, "Maximizing CNN accelerator efficiency through resource partitioning," in *Proc. ISCA*, 2017, pp. 535–547.
[22] S. Zheng, X. Zhang, L. Liu, S. Wei, and S. Yin, "Atomic dataflow based graph-level workload orchestration for scalable DNN accelerators," in *Proc. HPCA*, 2022, pp. 475–489.
[23] J. Cai, Y. Wei, Z. Wu, S. Peng, and K. Ma, "Inter-layer scheduling space definition and exploration for tiled accelerators," in *Proc. ISCA*, 2023, pp. 1–17.
[24] J. Cai et al., "Gemini: Mapping and architecture co-exploration for large-scale DNN chiplet accelerators," in *Proc. HPCA*, 2024, pp. 156–171.
[25] Z. Du et al., "ShiDianNao: Shifting vision processing closer to the sensor," in *Proc. ISCA*, 2015, pp. 92–104.
[26] D. Ignjatović and D. Capalija, "Scale-out first microarchitecture for efficient AI training," presented at Linley Spring Process. Conf., 2021. [Online]. Available: https://www.youtube.com/watch?v=Id3enIOAY2Q
[27] Z. Jia, S. Lin, C. R. Qi, and A. Aiken, "Exploring hidden dimensions in parallelizing convolutional neural networks," in *Proc. ICML*, 2018, pp. 2279–2288.
[28] M. Wang, C.-C. Huang, and J. Li, "Supporting very large models using automatic dataflow graph partitioning," in *Proc. EuroSys*, 2019, pp. 1–17.
[29] L. Ma et al., "Rammer: Enabling holistic deep learning compiler optimizations with $r$tasks," in *Proc. OSDI*, 2020, pp. 881–897.
[30] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, vol. 30, 2017, pp. 6000–6010.
[31] C. K. Williams and C. E. Rasmussen, *Gaussian Processes for Machine Learning*, vol. 2. Cambridge, MA, USA: MIT Press, 2006.
[32] R. L. Graham, "Bounds for certain multiprocessing anomalies," *Bell Syst. Tech. J.*, vol. 45, no. 9, pp. 1563–1581, 1966.
[33] Q. He, N. Guan, M. Lv, X. Jiang, and W. Chang, "Bounding the response time of DAG tasks using long paths," in *Proc. RTSS*, 2022, pp. 474–486.
[34] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM J. Appl. Math.*, vol. 17, no. 2, pp. 416–429, 1969.
[35] N. E. Jerger, T. Krishna, and L.-S. Peh, *On-Chip Networks* (Synthesis Lectures Computer Architecture), vol. 12. Cham, Switzerland: Springer, 2017, pp. 1–210.
[36] "Space-filling curve." 2023. [Online]. Available: https://en.wikipedia.org/wiki/Space-filling_curve
[37] H. Sagan, *Space-Filling Curves*. New York, NY, USA: Springer, 2012.
[38] W. Chen, X. Yao, X. Zhang, and B. Yu, "Efficient deep space filling curve," in *Proc. ICCV*, 2023, pp. 17525–17534.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.

[39] B. Moon, H. V. Jagadish, C. Faloutsos, and J. H. Saltz, "Analysis of the clustering properties of the Hilbert space-filling curve," *IEEE Trans. Knowl. Data Eng.*, vol. 13, no. 1, pp. 124–141, Jan./Feb. 2001.
[40] "Hilbert curve." 2024. [Online]. Available: https://en.wikipedia.org/wiki/Hilbert_curve
[41] H. V. Jagadish, "Analysis of the Hilbert curve for representing two-dimensional space," *Inf. Process. Lett.*, vol. 62, no. 1, pp. 17–22, 1997.
[42] C. Bai et al., "Klotski: DNN model orchestration framework for dataflow architecture accelerators," in *Proc. ICCAD*, 2023, pp. 1–9.
[43] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN Dataflows: A data-centric approach," in *Proc. MICRO*, 2019, pp. 754–768.
[44] "nn_dataflow: A neural network dataflow scheduling tool." Accessed: 2020. [Online]. Available: https://github.com/stanford-mast/nn_dataflow
[45] *Gurobi Optimizer Reference Manual (2020)*, Gurobi Optim. LLC., Beaverton, OR, USA, 2020.
[46] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, 2015, pp. 1–14. [Online]. Available: http://arxiv.org/abs/1409.1556
[47] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016, pp. 770–778.
[48] C. Szegedy et al., "Going deeper with convolutions," in *Proc. CVPR*, 2015, pp. 1–9.
[49] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL*, 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423

Chen Bai (Member, IEEE) received the B.E. degree in software engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2020, and the Ph.D. degree in computer science and engineering from the Chinese University of Hong Kong, Hong Kong, in 2024.

His research interests include computer architecture and electronic design automation.

Dr. Bai received the William J. McCalla Best Paper Award from ICCAD 2021 and the Best Paper Award Nomination from ISPD 2024.

Xuechao Wei received the Ph.D. degree from the Center for Energy-Efficient Computing and Applications, Peking University, Beijing, China, in 2019.

He is currently a Research Scientist with the Computing Technology Laboratory, Alibaba DAMO Academy, Beijing, China. His current research interests include high performance microprocessor and domain-specific architecture.

Youwei Zhuo (Member, IEEE) received the Ph.D. degree in computer science from the University of Southern California, Los Angeles, CA, USA, in 2021.

He is currently an Assistant Professor with the School of Integrated Circuits, Peking University, Beijing, China. His research interests include computer architecture and parallel programming, especially novel memory architecture.

Yi Cai received the B.S. and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2017 and 2022, respectively.

He is currently a Research Scientist with the Computing Technology Laboratory, Alibaba DAMO Academy, Shanghai, China. His research interests mainly include computer architecture and compiler technologies for deep learning.

Hongzhong Zheng (Member, IEEE) received the B.S. and M.S. degrees from Huazhong University of Science and Technology, Wuhan, China, in 1998 and 2001, respectively, and the Ph.D. degree in computer engineering from the University of Illinois at Chicago, Champaign, IL, USA, in 2009.

He was a Staff Memory Architecture with the Memory Solutions Laboratory, Semiconductor Inc., San Jose, CA, USA. He is currently a Research Scientist with the Computing Technology Laboratory, Alibaba DAMO Academy, Sunnyvale, CA, USA. His research interests include computer architecture, energy-efficient computing system designs, novel memory architecture emerging memory technology, and performance modeling.

Bei Yu (Senior Member, IEEE) received the Ph.D. degree from the University of Texas at Austin, Austin, TX, USA, in 2014.

He is currently an Associate Professor with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong.

Dr. Yu received ten Best Paper Awards from IEEE TSM 2022, DATE 2022, ICCAD 2021 and 2013, ASPDAC 2021 and 2012, ICTAI 2019, Integration VLSI Journal in 2018, ISPD 2017, SPIE Advanced Lithography Conference 2016, and many other awards, including the DAC Under-40 Innovator Award in 2024, the IEEE CEDA Ernest S. Kuh Early Career Award in 2022, and the Hong Kong RGC Research Fellowship Scheme Award in 2024. He has served as the TPC Chair for ACM/IEEE Workshop on Machine Learning for CAD, and in many journal editorial boards and conference committees.

Yuan Xie (Fellow, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, and the M.S. and Ph.D. degrees in computer engineering from Princeton University, Princeton, NJ, USA.

He is currently a Chair Professor with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong. He has a rich industry experience with IBM, Austin, TX, USA; AMD, Santa Clara, CA, USA; and Alibaba Group, Sunnyvale, CA. He was also on the faculty of Pennsylvania State University and University of California at Santa Barbara, Santa Barbara, CA, USA.

Dr. Xie is a recipient of many awards, including the NSF CAREER Award in 2006, the IEEE Computer Society Edward J. McCluskey Technical Achievement Award in 2021, and the IEEE CAS Society Industrial Pioneer Award in 2023. He is a Fellow of ACM and AAAS.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on October 22,2025 at 10:44:53 UTC from IEEE Xplore. Restrictions apply.